

明智地使用思考时间：用虚拟扩展加速 MCTS

Weirui Ye Pieter Abbeel Yang Gao
Tsinghua University UC Berkeley Shanghai Qi Zhi Institute

摘要

AI 研究中的一个核心问题，是如何在计算量与性能之间做取舍：理论上存在“完全理性”，但实践中不可能真正达到。近年来，蒙特卡洛树搜索（MCTS）因为能在多个挑战性领域显著提升性能而受到广泛关注。然而，搜索阶段昂贵的时间成本严重限制了它的应用范围。本文提出 Virtual MCTS（V-MCTS），它是 MCTS 的一个变体，能够自适应地在困难状态上花更多搜索时间，在简单状态上花更少搜索时间。我们给出该方法的理论界，并在 9×9 围棋和 Atari 游戏上评估性能与计算量。实验表明，该方法能达到与原始搜索算法相近的性能，同时平均只需要不到 50% 的搜索时间。我们认为，对于时间和资源受限的任务，这是一种可行替代方案。代码见 <https://github.com/YeWR/V-MCTS.git>。

1 引言

人工智能在 20 世纪 50 年代刚被系统研究时，研究者就开始追问：“如何构建具有完全理性的智能体”。这里的“完全理性”[7, 24, 26] 指在无限计算量下做出的决策。然而，经典搜索算法通常呈指数级运行时间；如果不考虑现实计算时间，人们只能解决小规模问题。因此，近年的 AI 研究不再追求“完全理性”，而是谨慎权衡计算量与理性程度。研究者提出了“有界最优性”等计算模型来描述这类设定 [26]。在相同计算预算下提升理性水平，已经带来许多 AI 成功案例，包括蒙特卡洛采样算法、变分推断算法，以及把 DNN 用作通用函数逼近器 [9, 8, 13, 30, 17]。

最近，基于 MCTS 的强化学习算法取得了巨大成功，主要体现在棋盘游戏中。最著名的成果是 AlphaGo 在 2015 年战胜樊麾 [30]，这是计算机程序首次击败人类职业围棋选手。随后，AlphaGo 又在 2016 年战胜李世石，并在 2017 年战胜当时世界排名第一的柯洁。之后，MCTS 强化学习算法进一步扩展到其他棋盘游戏和 Atari 游戏 [27]。EfficientZero [34] 显著提升了基于 MCTS 的 RL 算法样本效率，也为其未来在机器人和自动驾驶等真实世界场景中的应用带来希望。

尽管这类算法表现突出，它们训练和评估时需要巨大计算量。例如，MuZero [27] 使用 1000 个 TPU 训练 12 小时来学习围棋；对单个 Atari 游戏，也需要 40 个 TPU 训练 12 小时。

与 Atari 基准上的既有算法相比，它大约需要高两个数量级的计算。这种高昂计算需求既拖慢了 MCTS 强化学习算法自身的发展，也限制了它的实际使用。

从内部机制看，基于 MCTS 的 RL 算法会想象采取不同未来动作序列后的未来状态。但当前方法中的这种“想象”过程并不计算高效。例如，AlphaGo 为落下一颗棋子需要向前看 1600 个游戏状态；相反，顶级人类职业选手每分钟通常只能思考约 100–200 个游戏状态 [30]。除了低效外，当前 MCTS 对简单情形和困难情形使用相同计算预算；而人类知道应在最需要的时候使用时间。

本文旨在设计能节省 MCTS 型 RL 方法计算时间的新算法。我们的三个关键贡献如下：

1. 提出 Virtual MCTS，一种 MCTS 变体，可用更少计算近似 vanilla MCTS 的搜索策略。不同于以往主要作用于选择或评估阶段的剪枝方法，我们的方法改进搜索循环本身：当当前状态更简单时，自适应地提前终止搜索迭代。
2. 在理论上，我们为该方法提供误差界；可视化结果也显示 Virtual MCTS 比 vanilla MCTS 具有更好的计算-性能权衡。
3. 在经验上，我们的方法在有挑战性的 9×9 围棋上节省超过 50% 的搜索时间，在视觉复杂的 Atari 游戏上节省超过 60%，同时保持与 vanilla MCTS 相近的性能。

2 相关工作

2.1 结合 MCTS 的强化学习

长期以来，计算机围棋一直被视为非常困难的游戏 [3, 6]。研究者尝试使用蒙特卡洛技术，通过随机 playout 来评估节点状态价值 [4, 11, 12, 30]。随后，UCT 算法被广泛用于 MCTS 中，在树的每个节点使用 UCB1 选择动作 [20]。近年来，基于 MCTS 的 RL 方法 [30, 32, 31, 27] 越来越流行，并凭借强搜索能力在棋盘游戏上达到超人类表现。

现代 MCTS 强化学习算法的搜索循环包含四个阶段：选择、扩展、评估和回传。vanilla MCTS 的计算瓶颈来自搜索循环，尤其是每次迭代中的评估阶段和选择阶段。当搜索树更宽、更深时，选择阶段耗时增加；评估阶段也很昂贵，因为早期工作常通过随机 playout 到游戏结束来评估节点价值。由于搜索循环，相比 PPO [28]、SAC [16] 等无模型 RL 方法，MCTS 算法需要多次模型推理。

2.2 MCTS 加速

MCTS 方法已证明其解决复杂游戏或任务的强能力。但高计算成本阻碍了它进入实时和更一般场景，因此许多工作致力于加速 MCTS。

例如，为了提高选择阶段效率，一些启发式剪枝方法 [14, 33, 29, 1, 2] 使用启发函数减少搜索树的宽度和深度。为了更高效地评估，Lorentz [22] 提出 MCTS 的早期 *playout* 终止 (MCTS-EPT)，提前停止随机 *playout* 并用评估函数判断胜负。Hsueh 等 [18] 将 MCTS-EPT 应用于暗棋并证明其有效。类似思想后来也用于 AlphaGoZero [32] 及后续 MCTS 方法 [31, 27, 34] 的评估阶段：用可学习评估网络估计 Q 值，而不是 *playout* 到终局。Grill 等 [15] 从优化角度提出基于 AlphaZero 的正则化策略优化方法，以降低 MCTS 搜索预算。Danihelka 等 [10] 提出一种基于无放回采样动作的策略改进算法，即 Gumbel trick，在少量模拟规划时提升性能。

不过，上述方法主要关注搜索迭代中的某个具体阶段，或通过剪枝和优化方法减少总预算；这些方向与本文正交。直接针对搜索循环本身的工作很少。Lan 等 [21] 提出 DS-MCTS，在训练中用特定棋盘游戏特征和额外 DNN 近似 MCTS 的不确定性；评估时，DS-MCTS 会周期性检查，若状态足够确定则停止搜索。

3 背景

AlphaGo 系列工作 [30, 32, 31, 27] 都是基于 MCTS 的强化学习算法。这些算法假设环境转移动力学已知，或学习环境动力学。基于该动力学，它们把 MCTS 用作策略改进算子：输入当前策略，MCTS 通过搜索算法返回更好的策略。系统性搜索使 MCTS 强化学习算法能够快速改进策略，并在需要大量推理的场景中表现更好。

3.1 MCTS

本节简要介绍强化学习应用中的 MCTS。现代 MCTS 强化学习算法在搜索循环中包含四个阶段：选择、扩展、评估和回传。

MCTS 输入当前状态，在 N 次搜索循环后生成一个策略。这里 N 是设计者设定的常数迭代次数，可视为总预算。在每次迭代的选择阶段，动作通过最大化 UCB 选择。具体地，AlphaZero [31] 和 MuZero [27] 基于 UCB 的一个变体 P-UCT [25] 发展而来，并在棋盘游戏和 Atari 游戏上取得很大成功。P-UCT 公式为：

$$a_k = \arg \max_{a \in A} Q(s, a) + P(s, a) \frac{\sqrt{\sum_{b \in A} N(s, b)}}{1 + N(s, a)} \left(c_1 + \log \frac{\sum_{b \in A} N(s, b) + c_2 + 1}{c_2} \right).$$

其中 k 是迭代索引， A 是动作集合， $Q(s, a)$ 是估计 Q 值， $P(s, a)$ 是神经网络给出的策略先验， $N(s, a)$ 是从状态 s 选择动作 a 的访问次数， c_1, c_2 是超参数。

MCTS 的输出是根节点各动作的访问次数。经过 N 次搜索迭代后，最终策略 $\pi_N(s)$ 定义为归一化根访问分布： $\pi_k(s, a) = N(s, a) / \sum_{b \in A} N(s, b) = N(s, a) / k$ 。为简化记号，我们有时用 π_k 代替 $\pi_k(s)$ 。详细 MCTS 流程见附录。本文方法通过新的扩展方法和终止规则，用称为虚拟扩展策略的 $\hat{\pi}_k(s)$ 近似最终策略 $\pi_N(s)$ ，从而把 MCTS 迭代数从 N 降到 k 。

3.2 计算需求

MCTS 强化学习的大部分计算发生在 MCTS 过程中。MCTS 每采取一个动作都需要 N 次神经网络评估，其中 N 是搜索循环的常数迭代次数。传统 RL 算法如 PPO [28] 或 DQN [23] 每个动作只需要一次神经网络评估。因此，MCTS 强化学习粗略看比传统 RL 算法贵 N 倍。实践中，训练单个 Atari 游戏需要 40 个 TPU 运行 12 小时 [27]。尽管 MuZero 最终性能更好，其计算需求大约比传统 RL 算法 [28] 高两个数量级。

4 方法

我们的目标是在困难状态上花更多搜索时间，在简单状态上花更少搜索时间。直觉上，人类会根据情境决定快速决策还是慢速决策。但当前 MCTS 算法缺少这种情境感知行为。因此，我们提出一种可自适应终止搜索迭代的 MCTS 变体。它包含两个组件：一种名为虚拟扩展的新扩展方法，用当前部分树估计最终访问分布；以及一个根据当前场景困难程度决定何时终止的规则。第 5.5 节会通过可视化展示该自适应机制。

4.1 终止规则

我们提出基于当前树统计提前终止搜索循环。直觉上，如果最近若干次搜索对根访问分布几乎没有影响，就没有必要继续搜索。记 $\pi_k(s)$ 为第 k 次迭代时由根状态访问次数定义的策略。令 $\Delta_s(i, j)$ 为 $\pi_i(s)$ 与 $\pi_j(s)$ 的 L_1 差异，即 $\Delta_s(i, j) = \|\pi_i(s) - \pi_j(s)\|_1$ 。当已经搜索至少 rN 次且 $\Delta_s(k, k/2) < \epsilon$ 时，我们终止搜索；其中 ϵ 是容忍超参数， $r \in (0, 1)$ 是最小搜索预算比例， N 是完整搜索迭代数。我们证明，在一定条件下， $\Delta_s(k, k/2)$ 的界可推出 $\Delta_s(k, N)$ 的界。 $\Delta_s(k, N)$ 衡量当前策略 $\pi_k(s)$ 与 oracle 策略 $\pi_N(s)$ 的距离。这样， $\Delta_s(k, k/2)$ 反映了状态 s 的难度；一旦差距足够小，更多搜索就不再必要。

4.2 MCTS 中的虚拟扩展

终止规则 $\Delta_s(k, k/2) < \epsilon$ 默认 π_i 与 π_j 可直接比较。但由于树通过 UCT 扩展，二者并不能直接比较：访问次数越多，上界越紧，后期访问会更集中到有前景的部分。因此，较早的访问分布更具探索性，较晚的访问分布更偏向利用。

为正确比较 π_i 与 π_j ，我们用虚拟扩展替代 vanilla 扩展。简言之，它通过虚拟 UCT 扩展把两个分布都对齐到常数预算 N 。当树在第 k 次迭代时，距离完整预算还有 $N - k$ 次迭代。普通扩展会额外进行 $N - k$ 次神经网络评估，以便更准确估计根节点每个 arm 的 $Q(s, a)$ 。虚拟扩展仍按 UCT 扩展 $N - k$ 次，但忽略这些神经网络评估，并假设每个 arm 的 $Q(s, a)$ 不再改变。我们把从 π_i 得到的虚拟扩展分布记为 $\hat{\pi}_i$ 。对 π_i, π_j 都做虚拟扩展后，可得到 $\hat{\pi}_i, \hat{\pi}_j$ ，从而有效移除两个策略中不同程度的探索/利用差异。于是终止条件变为虚拟扩展策略之差：

$$\hat{\Delta}_s(k, k/2) = \|\hat{\pi}_k(s) - \hat{\pi}_{k/2}(s)\|_1 < \epsilon.$$

我们将该规则称为 VET-Rule (Virtual Expanded Termination Rule)。

算法 1: vanilla MCTS 的一次迭代

```

1 当前第  $k$  次迭代
  输入: 动作集  $A$ ; 先验  $P$ ; 估计  $Q_k(s, a)$ ; 访问次数  $N_k(s, a)$ 
  初始化:  $s \leftarrow s_{\text{root}}$ 
2 重复
3    $a^* \leftarrow \text{UCB1}(Q, P, N)$ 
4    $s \leftarrow \text{next}(s, a^*)$ 
5 直到  $N_k(s, a^*) = 0$ 
6 评估状态价值  $R(s, a)$  与先验  $P(s, a)$ 
7 对每个搜索路径上的  $(s, a)$  执行
8    $Q_{k+1}(s, a) \leftarrow \frac{N_k(s, a)Q_k(s, a) + R(s, a)}{N_k(s, a) + 1}$ 
9    $N_{k+1}(s, a) \leftarrow N_k(s, a) + 1$ 
10 返回  $Q_{k+1}(s, a), N_{k+1}(s, a)$ 
```

算法 2: 带虚拟扩展的 MCTS 迭代

```

1 当前第  $k$  次迭代
  输入: 动作集  $A$ ; 先验  $P$ ; 估计  $Q_k(s, a)$ ; 访问次数  $N_k(s, a)$ ; 虚拟
  访问次数  $\hat{N}_k(s, a)$ 
2 若  $\hat{N}_k(s, a)$  尚未初始化 则
  初始化:  $\hat{N}_k(s, a) \leftarrow N_k(s, a)$ 
3  $s \leftarrow s_{\text{root}}$ 
4  $a^* \leftarrow \text{UCB1}(Q, P, \hat{N})$ 
5  $\hat{N}_k(s, a^*) \leftarrow \hat{N}_k(s, a^*) + 1$ 
6 返回  $\hat{N}_k(s, a)$ 
```

vanilla 扩展和虚拟扩展的差异见算法 1 和算法 2。算法 1 中耗时计算主要来自：用 UCT 搜索到未访问状态、评估状态、沿路径回传以改进 Q 值估计。经过 N 次迭代后，根节点访问分布 $\pi_N(s)$ 被视为最终策略。而算法 2 中的虚拟扩展只从根节点搜索一步，并基于当前估计选择动作，不改变搜索树其他属性。虚拟访问计数 $\hat{N}_k(s, a)$ 会在虚拟访问后更新，用于平衡探索和利用。经过 $N - k$ 次虚拟扩展后，虚拟扩展策略为 $\hat{\pi}_k(s, a) = \hat{N}_k(s, a)/N$ ，而不是 $N_k(s, a)/k$ 。当 $k = N$ 时，根之后的进一步搜索不影响最终策略，因此 $\hat{\pi}_N(s, a) = \pi_N(s, a)$ 。

4.3 V-MCTS 算法

带 VET-Rule 的 MCTS 流程见算法 3。我们将该方法命名为 Virtual MCTS (V-MCTS)，即带 VET-Rule 的 MCTS 变体。与原始 MCTS 相比，算法中的终止判断只需进行少量低成本计算。一旦满足条件，搜索过程终止，并返回当前虚拟扩展策略 $\hat{\pi}_k(s)$ 。这样就跳过了后续 $N - k$ 次评估阶段中的神经网络模型预测。也就是说，我们用 $\hat{\pi}_k$ 近似 oracle 分布 π_N ，并把 N 次模拟预算降为 k 。其中 $k \geq rN$ ， r 是最小预算比例；最多可将树大小减少到约 $1/r$ 。

算法 3: Virtual MCTS

```

  输入: 预算  $N$ ; 根状态  $s$ ; 保守系数  $r$ ; 误差阈值  $\epsilon$ 
  说明:  $\pi_k(s)$  与  $\hat{\pi}_k(s)$  是策略分布;  $\pi_k(s, a)$  与  $\hat{\pi}_k(s, a)$  是动作  $a$  的概率。
1 对  $k = 1, 2, \dots, N$  执行
2   用 UCB1 选择动作
3   扩展新节点
4   用神经网络评估
5   回传并更新  $Q$  和访问次数
6    $\pi_k(s, a) \leftarrow N_k(s, a)/k$ 
7   虚拟扩展  $N - k$  个节点并更新  $\hat{N}_k(s, a)$ 
8    $\hat{\pi}_k(s, a) \leftarrow \hat{N}_k(s, a)/N$ 
9   若  $k \geq rN$  且  $\|\hat{\pi}_k(s) - \hat{\pi}_{k/2}(s)\|_1 < \epsilon$  则
10     $\pi(s) \leftarrow \hat{\pi}_k(s)$ 
11    跳出
12  $\pi(s) \leftarrow \pi_k(s)$ 
13 返回  $\pi(s)$ 
```

4.4 理论说明

我们进一步给出 V-MCTS 中 Q 值和 $\hat{\Delta}_s(k, N)$ 的理论界。记 k 为当前搜索迭代索引, N 为总搜索迭代数。 A 是动作集合且 $|A| > 1$ 。每个动作 $a \in A$ 关联一个取值在 $[0, 1]$ 的随机变量, 其期望为 Q_a 。第 k 步时, k 次试验上的经验均值为 $Q_k(s, a)$; 为简化记为 $\bar{Q}_k^a$ 。经过 $N - k$ 次虚拟扩展后的经验均值记为 $\bar{Q}_N^a$ 。由于只处理根节点访问分布, 我们省略根状态下标。为方便起见, 假设不同动作按期望值排序: $1 \geq Q_1 \geq Q_2 \geq \dots \geq Q_{|A|} \geq 0$ 。

定理 4.1. 给定 $r \in (0, 1)$ 、置信参数 $\delta \in (0, 1)$ 和有限动作集 A , 存在 $N_0 > 0$, 使得对所有 $N > N_0$ 与 $k \geq rN$, 令 $\epsilon_k = \sqrt{\ln(100k^2/\delta)/(2k)}$, 在 k 次 vanilla 扩展和 $N - k$ 次虚拟扩展后, 有: (a) 虚拟扩展中的价值一致性; (b) 虚拟扩展中的最优动作识别。也就是说, 足够多次试验后, 各动作期望 Q 值能由虚拟扩展后的经验 Q 值估计, 而最佳经验动作的 Q 值以高概率接近最佳期望动作的 Q 值。

定理 4.2 (V-MCTS 误差界). 给定 $r \in (0, 1)$ 、置信参数 $\delta \in (0, 1)$ 和有限动作集 A 。若虚拟扩展策略 $\hat{\pi}_k$ 由算法 3 生成, 则存在 $N_0 > 0$, 使得对所有 $N > N_0$ 、 $k \geq rN$ 和任意 $\epsilon \in (0, 1]$, 若 $\|\hat{\pi}_k(s) - \hat{\pi}_{k/2}(s)\|_1 < \epsilon$, 则以高概率有 $\|\pi_N(s) - \hat{\pi}_k(s)\|_1 < 3\epsilon$ 。

定理 4.2 表明, $\hat{\Delta}_s(k, k/2)$ 的界可高概率推出 $\hat{\Delta}_s(k, N)$ 的界。因此, 当试验足够多时, oracle 策略 $\pi_N(s)$ 可由 $\hat{\pi}_k(s)$ 近似。给定最小距离 ϵ , 对较容易状态, 规则 $\hat{\Delta}_s(k, k/2) < \epsilon$ 更容易满足, 因为即使搜索迭代增加, 树节点的 Q 值也保持在较小范围内。因此, V-MCTS 生成的虚拟扩展策略接近 oracle 策略, 并会在状态更容易时更早终止搜索循环。

5 实验

本节实验目标是证明 V-MCTS 的有效性 & 效率。首先, 我们比较 vanilla MCTS 与本文方法的性能和成本, 具体评估 9×9 围棋和若干 Atari 游戏。其次, 我们进行消融实验, 以检验虚拟扩展的有效性和超参数敏感性。最后, 通过可视化和性能分析展示自适应机制。

5.1 设置

模型和环境. Ye 等 [34] 提出了 EfficientZero, 这是 MuZero [27] 的一个变体, 通过三个额外组件提升样本效率, 训练只需要 8 个 GPU, 因此更可负担。本文选择 9×9 围棋和若干 Atari 游戏作为基准环境。围棋用于测试算法在挑战性规划问题中的表现; Atari 游戏则具有视觉复杂性。

超参数. 对 9×9 围棋, 采用 Tromp-Taylor 规则。围棋环境基于开源代码库 GymGo [19] 构建。我们让智能体与 GNU Go v3.8 level 10 [5] 对弈 200 局评估性能, 其中 100 局执黑、100 局执白, 并使用不同随机种子。贴目设为 6.5。对 Atari 游戏, 选择 5 个游戏, 使用 100k 环境步。每个设定使用 3 个训练种子, 并为每个训练模型使用 100 个评估种子。

基线. 我们在 9×9 围棋和 Atari 游戏上, 将本文方法与带 vanilla MCTS 的 EfficientZero 比较。DS-MCTS [21] 也通过训练出的不确定性网络自适应终止 MCTS, 但它需要为围棋设计的特定特征, 且只适用于评估阶段。因此, 我们也在围棋上与 DS-MCTS 比较最终性能。

5.2 围棋结果

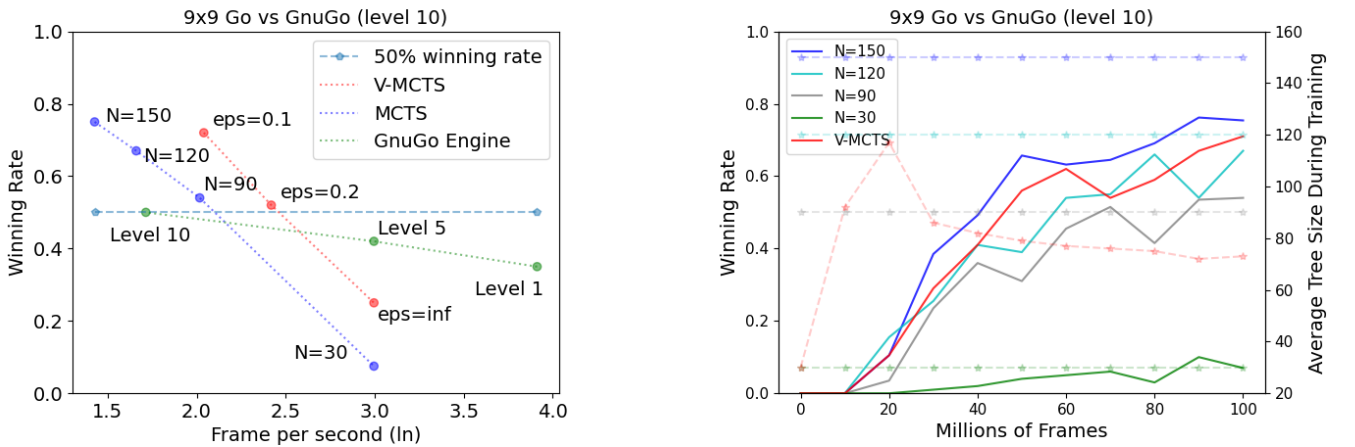


图 1: V-MCTS 在 9×9 围棋中对 GNU Go (level 10) 的表现。(a) 比较 MCTS、V-MCTS 和不同 level 的 GNU Go 的速度与胜率。V-MCTS 具有更好的计算-性能权衡。横轴为对数尺度下的 FPS。(b) 训练阶段的胜率 and 平均树大小; 实线为胜率, 虚线为树大小。红色为 V-MCTS, 其他为不同 N 的 vanilla MCTS。

图 1(a) 展示了与同一 GnuGo (level 10) 智能体对弈时的计算-性能权衡。横轴为训练速度, 纵轴为胜率; 越靠左上, 性能与计算权衡越好。我们用常数预算 N 训练基线方法 (蓝点), 并训练 $r = 0.2, \epsilon = 0.1$ 的 V-MCTS。对训练后模型, 我们用不同 ϵ 展

示计算与性能的权衡（红点）。绿点为不同 level 的 GnuGo。GnuGo 引擎提供 level 1–10 的模型，每个 level 对应运行时间和强度之间的一种权衡；纵轴为对 level 10 的胜率。

首先，对所有方法而言，更多搜索迭代（更大的 N 或更小的 ϵ s）带来更高胜率，但响应时间也更高。其次，V-MCTS ($\epsilon = 0.1$) 对 GnuGo level 10 达到 71% 胜率，接近 MCTS ($N = 150$) 的 75%。V-MCTS ($\epsilon = 0.1$) 单步落子的时间成本为 0.12s，而 GnuGo 为 0.18s，MCTS ($N = 150$) 超过 0.2s。因此，该终止规则能用更少预算保持强性能。图 1(b) 展示训练阶段胜率和平均树大小变化。红色虚线显示，V-MCTS 的树大小在训练中变化，并持续小于最大尺寸 ($N = 150$)，同时胜率与 MCTS ($N = 150$) 相近。因此，V-MCTS 不仅在评估中有效，也能在训练中工作。

	MCTS ($N = 150$)	DS-MCTS	V-MCTS
平均预算	150 ± 0.0	97 ± 12.5	76 ± 10.8
胜率	$75\% \pm 3.0\%$	$60\% \pm 4.0\%$	$71\% \pm 4.7\%$

表 1：9 × 9 围棋结果：3 次独立训练、200 局对弈上的胜率和平均预算比较。V-MCTS 在预算与胜率两方面均优于 DS-MCTS。

MCTS	$N = 50$	$N = 30$	$N = 10$	V-MCTS	预算 k
Pong	19.7 ± 1.6	12.5 ± 5.5	2.0 ± 1.3	18.8 ± 2.8	13.3 ± 0.6
Breakout	410.7 ± 15.1	370.9 ± 34.1	303.9 ± 11.3	372.8 ± 18.3	15.7 ± 0.6
Seaquest	1159.9 ± 90.7	775.2 ± 146.8	555.4 ± 66.9	970.0 ± 339.5	14.3 ± 1.2
Hero	9992.1 ± 2059.4	9241.3 ± 3615.3	4437.0 ± 2490.6	8928.1 ± 2922.1	15.0 ± 1.0
Qbert	14495.8 ± 683.9	10429.9 ± 2291.1	8149.8 ± 2085.0	11476.6 ± 978.2	16.3 ± 1.2

表 2：Atari 游戏结果：3 次独立训练、100 个评估种子上的得分。 $N = 50$ 是 oracle MCTS。除 oracle 外，粗体为最好结果。V-MCTS 取得更好的性能–计算权衡。

5.3 Atari 结果

除了围棋，我们还在视觉复杂的游戏上评估方法。由于 Atari 游戏的搜索空间比围棋小且更容易，我们选择若干 Atari 游戏来研究方法对性能的影响。实验遵循 EfficientZero 的 100k Atari benchmark 设置，即只包含 400k 帧数据。表 2 显示了结果。总体看，本文方法在 Atari 游戏上有效。树大小是自适应的，V-MCTS 的性能仍与完整搜索 MCTS 相近。它比 MCTS ($N = 30$) 表现更好，同时需要更少搜索，证明了方法的有效性和效率。Hero 是一个例外，但 V-MCTS 的表现仍非常接近 vanilla MCTS ($N = 30$)，而平均只使用约一半搜索迭代。此外，Atari 上搜索次数下降幅度比围棋更大。

总之，V-MCTS 能在更少搜索迭代下保持相近性能；若只是简单减少 MCTS 总预算，则性能下降更明显。并且，在较容易环境中，搜索成本节省更显著。

5.4 消融研究

前述结果说明，本文方法能降低 MCTS 响应时间，同时在挑战性任务上保持相近性能。本节分析哪些组件贡献了性能，以及超参数如何影响结果；也消融 VET-Rule 中不同范数准则和更大 MCTS 预算 N 的影响。

虚拟扩展。为验证虚拟扩展有效性，我们将其与另外两种扩展方法比较：一种是算法 1 中的 vanilla 扩展，即在第 k 次迭代返回并输出 π_k ；另一种是 greedy 扩展，把剩余 $N - k$ 次模拟都贪心地用于搜索当前最佳动作。我们在 $k = 30$ 时停止搜索，其中 $k = rN$ 、 $r = 0.2$ 、 $N = 150$ ，随后分别做虚拟扩展、贪心扩展或不做额外操作。表 3 显示，虚拟扩展胜率达到了 32%，显著好于其他方法；贪心扩展因过度利用导致严重探索问题。

算法	平均大小	胜率
Vanilla expansion	30	$17\% \pm 3.2\%$
Greedy expansion	30	$3\% \pm 2.0\%$
Virtual expansion	30	$32\% \pm 3.5\%$

表 3：9 × 9 围棋中不同扩展方法的消融结果，统计自 3 次独立训练。虚拟扩展能在探索与利用之间取得更好平衡。

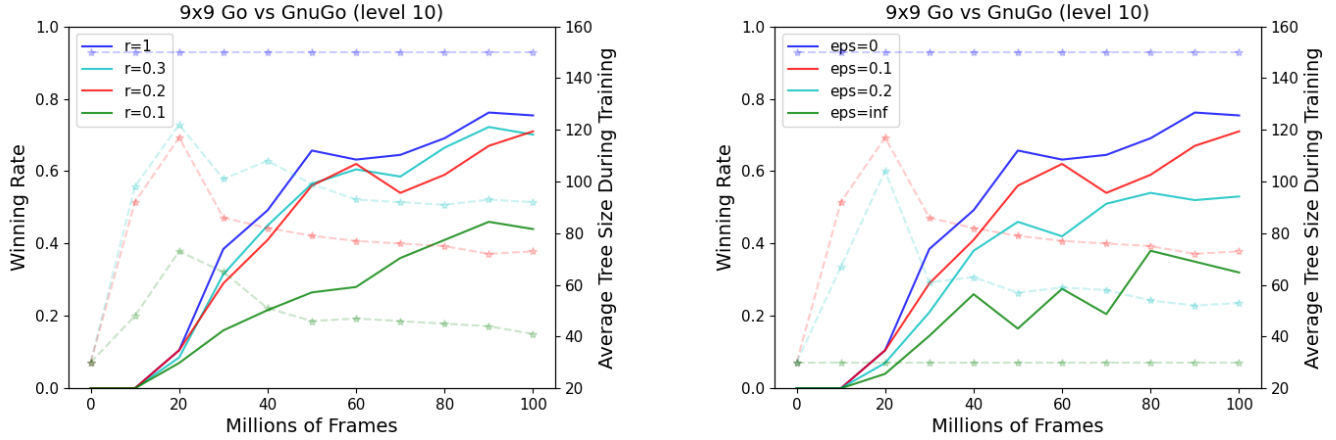


图 2: VET-Rule 对超参数 r, ϵ 的敏感性。实线表示胜率, 虚线表示平均树大小; 该图说明 V-MCTS 能通过终止阈值在计算量与性能之间调节。

终止规则. 为在尽量降低树规模的同时保持性能, 需要寻找更好的终止规则。如第 4.1 节所述, VET-Rule 有两个超参数 r, ϵ 。其中 r 是最小预算比例 rN , ϵ 是最小距离 $\hat{\Delta}_s(k, k/2)$ 的阈值。我们分别消融 r 与 ϵ 的不同取值, 默认值设为 $r = 0.2, \epsilon = 0.1$ 。

图 2 比较了训练阶段的胜率和平均树大小。首先, 当 $r \geq 0.2$ 时, 胜率对 r 并不敏感; 但平均树大小对 r 敏感, 因为 V-MCTS 至少要搜索 rN 次。此外, $r = 0.1$ 与 $r = 0.2$ 之间存在性能下降。因此, $r = 0.2$ 是速度与性能之间较合理的折中。其次, ϵ 越大, $\hat{\Delta}_s(k, k/2) < \epsilon$ 越容易满足, 树规模也越小; 但胜率通常随 ϵ 变大而下降。实践中应选择合适的最小距离, 在计算成本和策略质量之间折中。

VET-Rule 中的范数准则. 本文采用 $\|\hat{\pi}_k(s) - \hat{\pi}_{k/2}(s)\|_1 < \epsilon$ 作为终止条件。 L_2 范数也可以放大较大的偏差, 因此我们比较 L_1 与 L_2 两种策略。表 4 显示, L_2 也能工作, 但 L_1 效果更好。原因可能在于 UCB 分数已经通过 $N(s, a)$ 考虑访问次数差异; 再放大偏差可能引入偏置。

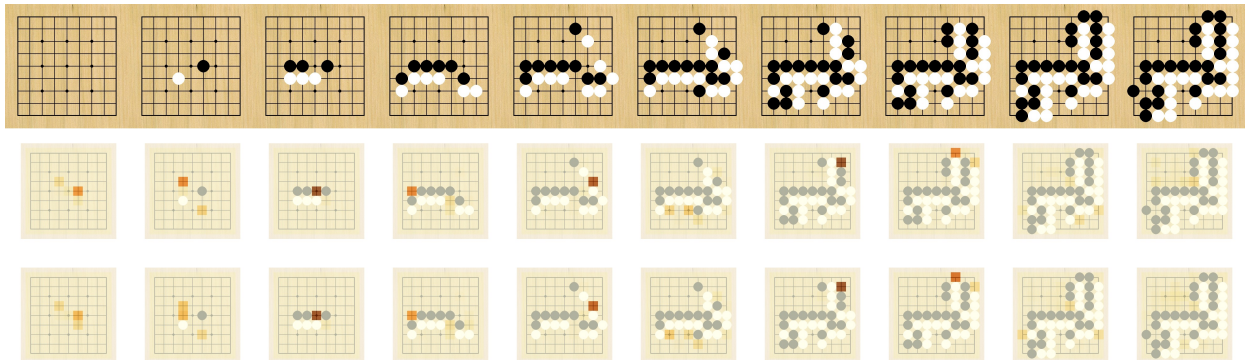
方法	平均预算	胜率
MCTS ($N = 150$)	150	82.0%
V-MCTS $L_1, N = 150, r = 0.2, \epsilon = 0.1$	96.2	81.5%
V-MCTS $L_2, N = 150, r = 0.2, \epsilon = 0.1$	97.1	79.8%
V-MCTS $L_2, N = 150, r = 0.2, \epsilon = 0.05$	119.3	81.0%

表 4: VET-Rule 中不同范数策略的胜率与平均预算比较。

更大的 MCTS 预算. 为验证方法在更大搜索预算下是否仍有效, 我们比较 $N = 150/400/600/800$ 的 vanilla MCTS, 以及 $N = 800, r = 0.2, \epsilon = 0.1$ 的 V-MCTS。结果见表 5。V-MCTS 在平均预算和胜率上优于 MCTS($N = 600$), 并能以更少平均预算接近 oracle MCTS($N = 800$)。

MCTS	$N = 150$	$N = 400$	$N = 600$	$N = 800$	V-MCTS
平均预算	150	400	600	800	431.1
胜率	82.0%	84.5%	84.9%	85.9%	85.0%

表 5: 大预算 MCTS 扩展下的胜率与平均预算比较。



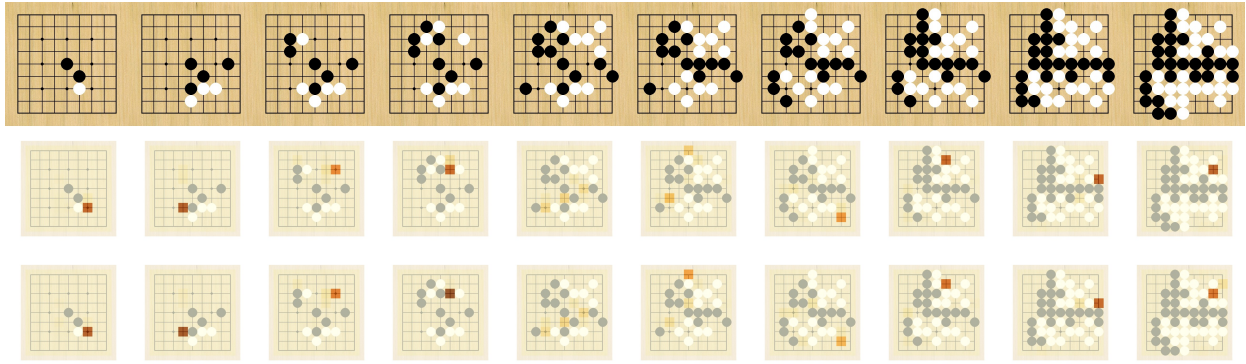


图 3: MCTS($N = 150$) 与 V-MCTS 的策略分布热力图。上图为执黑, 下图为执白; 更深的红色表示相应动作访问次数更高。V-MCTS 会根据局面在不同搜索时间 k 终止, 并生成接近 oracle 的策略分布。

5.5 V-MCTS 自适应行为的可视化

本节通过可视化和性能分析展示自适应机制。我们发现: (1) $\hat{\pi}_k(s)$ 接近 $\pi_N(s)$; (2) 面对更简单状态时, V-MCTS 会更早终止。具体而言, 我们选取训练后模型与 GnuGo 对弈过程中不同时间步的围棋局面, 并在图 3 中展示策略分布热力图。每个子图最后两行分别是 oracle MCTS(π_N) 与 V-MCTS 在满足 $\hat{\Delta}_s(k, k/2) < \epsilon$ 时得到的 $\hat{\pi}_k$ 。棋盘上颜色越深, 表示搜索中相应动作被访问得越多。总体上, 不同状态中的 $\hat{\pi}_k$ 与 π_N 接近, 说明虚拟扩展后的策略分布能逼近 oracle 策略。

此外, 可选价值动作越少, V-MCTS 越早终止。以围棋为例, 开局通常并不复杂, 因为棋盘上只有少数棋子; 收官阶段情况更复杂。图中可见, 开局列的终止更早, 而复杂局面的终止更晚。更重要的是, 终止步数 k 并不只是由棋子数量决定, 而是由当前局面的搜索稳定性决定。因此, V-MCTS 能根据局面自适应终止搜索, 并生成接近 oracle 的策略; 在简单状态上更早停下, 从而取得更好的计算-性能折中。

6 讨论

本文提出 V-MCTS, 用于加速 MCTS 并决定搜索迭代何时终止。该方法能在保持相近表现的同时, 自适应减少约一半搜索时间。我们认为这为把基于 MCTS 的方法应用到实时场景迈出了一步。本文的一个局限是尚不能处理连续动作空间环境。未来工作将考虑把早停机制扩展到连续动作空间。

致谢与经费披露

本研究得到中华人民共和国科学技术部 2030 创新重大项目“新一代人工智能”计划支持 (Grant No. 2021AAA0150000)。本研究还得到清华大学国强研究院资助。

References

- [1] H. Baier and M. H. Winands. MCTS-minimax hybrids. *IEEE Transactions on Computational Intelligence and AI in Games*, 7(2):167–179, 2014.
- [2] H. Baier and M. H. Winands. MCTS-minimax hybrids with state evaluations. *Journal of Artificial Intelligence Research*, 62:193–231, 2018.
- [3] B. Bouzy and T. Cazenave. Computer go: an AI oriented survey. *Artificial Intelligence*, 132(1):39–103, 2001.
- [4] B. Bouzy and B. Helmstetter. Monte-Carlo go developments. In *Advances in Computer Games*, pages 159–174. Springer, 2004.
- [5] D. Bump, M. L. Li, W. Iba, and et al. GnuGo, 2005.
- [6] X. Cai and D. C. Wunsch. Computer go: A grand challenge to AI. *Challenges for Computational Intelligence*, pages 443–465, 2007.
- [7] R. Carnap. *Logical Foundations of Probability*. 1962.
- [8] G. Chaslot, S. Bakkes, I. Szita, and P. Spronck. Monte-Carlo tree search: A new framework for game AI. *AIIDE*, 8:216–217, 2008.
- [9] R. Coulom. Efficient selectivity and backup operators in Monte-Carlo tree search. In *International Conference on Computers and Games*, pages 72–83. Springer, 2006.
- [10] I. Danihelka, A. Guez, J. Schrittwieser, and D. Silver. Policy improvement by planning with Gumbel. In *International Conference on Learning Representations*, 2022.
- [11] S. Gelly and D. Silver. Combining online and offline knowledge in UCT. In *Proceedings of the 24th International Conference on Machine Learning*, pages 273–280, 2007.
- [12] S. Gelly and D. Silver. Achieving master level play in 9 × 9 computer go. In *AAAI*, volume 8, pages 1537–1540, 2008.
- [13] S. Gelly and D. Silver. Monte-Carlo tree search and rapid action value estimation in computer go. *Artificial Intelligence*, 175(11):1856–1875, 2011.
- [14] S. Gelly, Y. Wang, R. Munos, and O. Teytaud. Modification of UCT with patterns in Monte-Carlo Go. PhD thesis, INRIA, 2006.
- [15] J.-B. Grill, F. Altche, Y. Tang, T. Hubert, M. Valko, I. Antonoglou, and R. Munos. Monte-Carlo tree search as regularized policy optimization. In *International Conference on Machine Learning*, pages 3769–3778. PMLR, 2020.
- [16] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International Conference on Machine Learning*, pages 1861–1870. PMLR, 2018.
- [17] M. D. Hoffman, D. M. Blei, C. Wang, and J. Paisley. Stochastic variational inference. *Journal of Machine Learning Research*, 14(5), 2013.
- [18] C.-H. Hsueh, I.-C. Wu, W.-J. Tseng, S.-J. Yen, and J.-C. Chen. An analysis for strength improvement of an MCTS-based program playing Chinese dark chess. *Theoretical Computer Science*, 644:63–75, 2016.
- [19] E. Huang. GymGo. <https://github.com/aigagror/GymGo>, 2021.
- [20] L. Kocsis and C. Szepesvari. Bandit based Monte-Carlo planning. In *European Conference on Machine Learning*, pages 282–293. Springer, 2006.

- [21] L.-C. Lan, M.-Y. Tsai, T.-R. Wu, I. Wu, C.-J. Hsieh, et al. Learning to stop: Dynamic simulation Monte-Carlo tree search. *arXiv preprint arXiv:2012.07910*, 2020.
- [22] R. Lorentz. Early playout termination in MCTS. In *Advances in Computer Games*, pages 12–19. Springer, 2015.
- [23] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.
- [24] A. Newell. The knowledge level. *Artificial Intelligence*, 18(1):87–127, 1982.
- [25] C. D. Rosin. Multi-armed bandits with episode context. *Annals of Mathematics and Artificial Intelligence*, 61(3):203–230, 2011.
- [26] S. J. Russell and D. Subramanian. Provably bounded-optimal agents. *Journal of Artificial Intelligence Research*, 2:575–609, 1994.
- [27] J. Schrittwieser, I. Antonoglou, T. Hubert, K. Simonyan, L. Sifre, S. Schmitt, A. Guez, E. Lockhart, D. Hassabis, T. Graepel, et al. Mastering Atari, Go, chess and shogi by planning with a learned model. *Nature*, 588(7839):604–609, 2020.
- [28] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [29] N. Sephton, P. I. Cowling, E. Powley, and N. H. Slaven. Heuristic move pruning in Monte Carlo tree search for the strategic card game Lords of War. In *2014 IEEE Conference on Computational Intelligence and Games*, pages 1–7. IEEE, 2014.
- [30] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. Van Den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, et al. Mastering the game of Go with deep neural networks and tree search. *Nature*, 529(7587):484–489, 2016.
- [31] D. Silver, T. Hubert, J. Schrittwieser, I. Antonoglou, M. Lai, A. Guez, M. Lanctot, L. Sifre, D. Kumaran, T. Graepel, et al. A general reinforcement learning algorithm that masters chess, shogi, and go through self-play. *Science*, 362(6419):1140–1144, 2018.
- [32] D. Silver, J. Schrittwieser, K. Simonyan, I. Antonoglou, A. Huang, A. Guez, T. Hubert, L. Baker, M. Lai, A. Bolton, et al. Mastering the game of Go without human knowledge. *Nature*, 550(7676):354–359, 2017.
- [33] Y. Wang and S. Gelly. Modifications of UCT and sequence-like simulations for Monte-Carlo Go. In *2007 IEEE Symposium on Computational Intelligence and Games*, pages 175–182. IEEE, 2007.
- [34] W. Ye, S. Liu, T. Kurutach, P. Abbeel, and Y. Gao. Mastering Atari games with limited data. In *NeurIPS*, 2021.